educative

Introduction

0/1 Knapsack pattern is based on the famous problem with the same name which is efficiently solved using **Dynamic Programming (DP)**.

In this pattern, we will go through a set of problems to develop an understanding of DP. We will always start with a brute-force recursive solution to see the overlapping subproblems, i.e., realizing that we are solving the same problems repeatedly.

After the recursive solution, we will modify our algorithm to apply advanced techniques of **Memoization** and **Bottom-Up Dynamic Programming** to develop a complete understanding of this pattern.

Let's jump onto our first problem.

Back

Mark As Completed

Next

Solution Review: Problem Challenge 1

0/1 Knapsack (medium)

0/1 Knapsack (medium)

We'll cover the following

- Introduction
- Problem Statement
- Try it yourself
- Basic Solution
  - Code
  - Time and Space complexity
- Top-down Dynamic Programming with Memoization
  - Code
  - Time and Space complexity
- Bottom-up Dynamic Programming
  - Code
  - Time and Space complexity
  - How can we find the selected items?
- Challenge

Introduction

Given the weights and profits of 'N' items, we are asked to put these items in a knapsack with a capacity 'C.' The goal is to get the maximum profit out of the knapsack items. Each item can only be selected once, as we don't have multiple quantities of any item.

Let's take Merry's example, who wants to carry some fruits in the knapsack to get maximum profit. Here are the weights and profits of the fruits:

Items: { Apple, Orange, Banana, Melon }
Weights: { 2, 3, 1, 4 }
Profits: { 4, 5, 3, 7 }
Knapsack capacity: 5

Let's try to put various combinations of fruits in the knapsack, such that their total weight is not more than 5:

Apple + Orange (total weight 5) => 9 profit
Apple + Banana (total weight 3) => 7 profit
Orange + Banana (total weight 4) => 8 profit
Banana + Melon (total weight 5) => 10 profit

This shows that **Banana** + **Melon** is the best combination as it gives us the maximum profit, and the total weight does not exceed the capacity.

Problem Statement

Given two integer arrays to represent weights and profits of 'N' items, we need to find a subset of these items which will give us maximum profit such that their cumulative weight is not more than a given number 'C.' Each item can only be selected once, which means either we put an item in the knapsack or we skip it.

Try it yourself

Try solving this question here:

Java | Python3 | C++ | JS

```
class Knapsack {

  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    // TODO: Write your code here
    return -1;
  }

  public static void main(String[] args) {
    Knapsack ks = new Knapsack();
    int[] profits = {1, 6, 10, 16};
    int[] weights = {1, 2, 3, 5};
    int maxProfit = ks.solveKnapsack(profits, weights, 7);
    System.out.println("Total knapsack profit ---> " + maxProfit);
    maxProfit = ks.solveKnapsack(profits, weights, 6);
    System.out.println("Total knapsack profit ---> " + maxProfit);
  }
}
```

Run | Save | Reset

Basic Solution

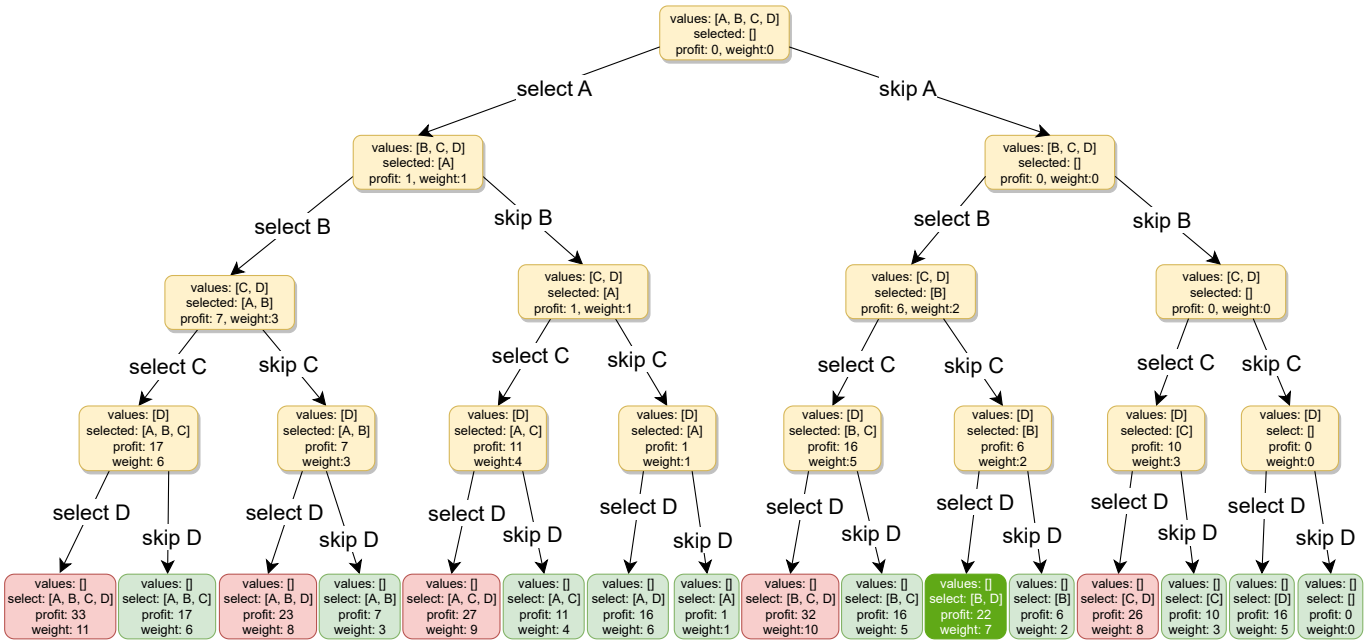
A basic brute-force solution could be to try all combinations of the given items (as we did above), allowing us to choose the one with maximum profit and a weight that doesn't exceed 'C'. Take the example of four items (A, B, C, and D), as shown in the diagram below. To try all the combinations, our algorithm will look like:

```
for each item 'i'
  create a new set which INCLUDES item 'i' if the total weight does not exceed the capacity, and
     recursively process the remaining capacity and items
  create a new set WITHOUT item 'i', and recursively process the remaining items
return the set from the above two sets with higher profit
```

Here is a visual representation of our algorithm:

items	A	B	C	D
profit	1	6	10	16
weight	1	2	3	5

Capacity: 7



All green boxes have a total weight that is less than or equal to the capacity (7), and all the red ones have a weight that is more than 7. The best solution we have is with items [B, D] having a total profit of 22 and a total weight of 7.

Code

Here is the code for the brute-force solution:

Java | Python3 | C++ | JS

```
class Knapsack {

  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    return this.knapsackRecursive(profits, weights, capacity, 0);
  }

  private int knapsackRecursive(int[] profits, int[] weights, int capacity, int currentIndex) {
    // base checks
    if (capacity <= 0 || currentIndex >= profits.length)
      return 0;

    // recursive call after choosing the element at the currentIndex
    // if the weight of the element at currentIndex exceeds the capacity, we shouldn't process this
    int profit1 = 0;
    if( weights[currentIndex] <= capacity )
        profit1 = profits[currentIndex] + knapsackRecursive(profits, weights,
                capacity - weights[currentIndex], currentIndex + 1);

    // recursive call after excluding the element at the currentIndex
    int profit2 = knapsackRecursive(profits, weights, capacity, currentIndex + 1);

    return Math.max(profit1, profit2);
  }

  public static void main(String[] args) {
    Knapsack ks = new Knapsack();
    int[] profits = {1, 6, 10, 16};
```

```
28      int[] weights = {1, 2, 3, 5};
29      int maxProfit = ks.solveKnapsack(profits, weights, 7);
30      System.out.println("Total knapsack profit ---> " + maxProfit);
31      maxProfit = ks.solveKnapsack(profits, weights, 6);
32      System.out.println("Total knapsack profit ---> " + maxProfit);
33    }
34  }
```

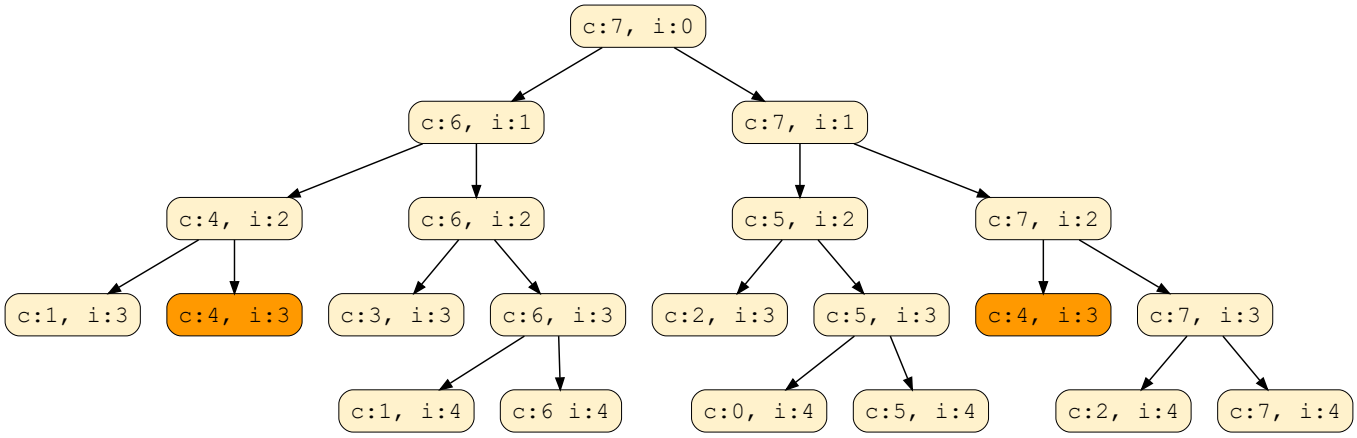
Run Save Reset

Time and Space complexity

The above algorithm's time complexity is exponential $O(2^n)$, where 'n' represents the total number of items. This can also be confirmed from the above recursion tree. As we can see, we will have a total of '31' recursive calls – calculated through $(2^n) + (2^n) - 1$, which is asymptotically equivalent to $O(2^n)$.

The space complexity is $O(n)$. This space will be used to store the recursion stack. Since the recursive algorithm works in a depth-first fashion, which means that we can't have more than 'n' recursive calls on the call stack at any time.

Overlapping Sub-problems: Let's visually draw the recursive calls to see if there are any overlapping sub-problems. As we can see, in each recursive call, profits and weights arrays remain constant, and only capacity and currentIndex change. For simplicity, let's denote capacity with 'c' and currentIndex with 'i':



We can clearly see that **'c:4, i=3'** has been called twice. Hence we have an overlapping sub-problems pattern. We can use Memoization to solve overlapping sub-problems efficiently.

Top-down Dynamic Programming with Memoization

Memoization is when we store the results of all the previously solved sub-problems and return the results from memory if we encounter a problem that has already been solved.

Since we have two changing values (`capacity` and `currentIndex`) in our recursive function `knapsackRecursive()`, we can use a two-dimensional array to store the results of all the solved sub-problems. As mentioned above, we need to store results for every sub-array (i.e., for every possible index 'i') and every possible capacity 'c.'

Code

Here is the code with memoization (see changes in the highlighted lines):

Java | Python3 | C++ | JS

```java
class Knapsack {

  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    Integer[][] dp = new Integer[profits.length][capacity + 1];
    return this.knapsackRecursive(dp, profits, weights, capacity, 0);
  }

  private int knapsackRecursive(Integer[][] dp, int[] profits, int[] weights, int capacity,
      int currentIndex) {

    // base checks
    if (capacity <= 0 || currentIndex >= profits.length)
      return 0;

    // if we have already solved a similar problem, return the result from memory
    if(dp[currentIndex][capacity] != null)
      return dp[currentIndex][capacity];

    // recursive call after choosing the element at the currentIndex
    // if the weight of the element at currentIndex exceeds the capacity, we shouldn't process this
    int profit1 = 0;
    if( weights[currentIndex] <= capacity )
        profit1 = profits[currentIndex] + knapsackRecursive(dp, profits, weights,
                capacity - weights[currentIndex], currentIndex + 1);

    // recursive call after excluding the element at the currentIndex
    int profit2 = knapsackRecursive(dp, profits, weights, capacity, currentIndex + 1);

    dp[currentIndex][capacity] = Math.max(profit1, profit2);
    return dp[currentIndex][capacity];
  }

  public static void main(String[] args) {
    Knapsack ks = new Knapsack();
    int[] profits = {1, 6, 10, 16};
    int[] weights = {1, 2, 3, 5};
    int maxProfit = ks.solveKnapsack(profits, weights, 7);
    System.out.println("Total knapsack profit ---> " + maxProfit);
    maxProfit = ks.solveKnapsack(profits, weights, 6);
    System.out.println("Total knapsack profit ---> " + maxProfit);
  }
}
```

Run | Save | Reset

Time and Space complexity

Since our memoization array `dp[profits.length][capacity+1]` stores the results for all subproblems, we can conclude that we will not have more than $N * C$ subproblems (where 'N' is the number of items and 'C' is the knapsack capacity). This means that our time complexity will be $O(N * C)$.

The above algorithm will use $O(N * C)$ space for the memoization array. Other than that, we will use $O(N)$ space for the recursion call-stack. So the total space complexity will be $O(N * C + N)$, which is asymptotically equival to $O(N * C)$.

Bottom-up Dynamic Programming

Let's try to populate our `dp[][]` array from the above solution by working in a bottom-up fashion. Essentially, we want to find the maximum profit for every sub-array and every possible capacity. **This means that `dp[i][c]` will represent the maximum knapsack profit for capacity 'c' calculated from the first 'i' items.**

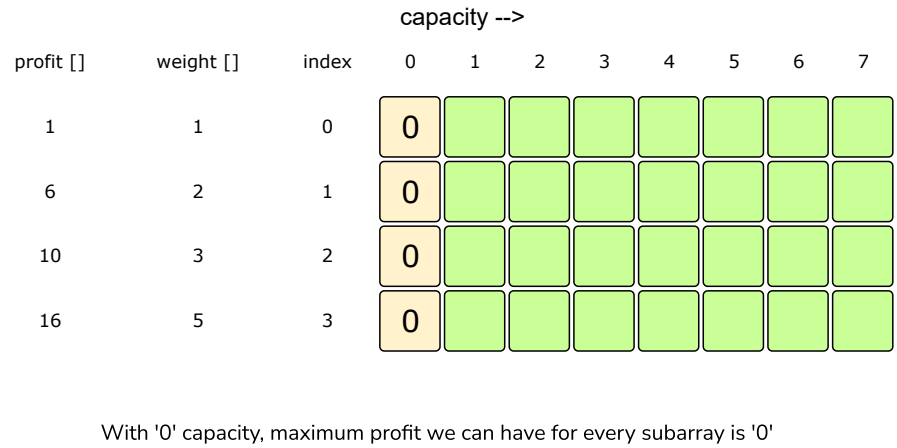
So, for each item at index 'i' (0 <= i < items.length) and capacity 'c' (0 <= c <= capacity), we have two options:

1. Exclude the item at index 'i.' In this case, we will take whatever profit we get from the sub-array excluding this item => `dp[i-1][c]`
2. Include the item at index 'i' if its weight is not more than the capacity. In this case, we include its profit plus whatever profit we get from the remaining capacity and from remaining items => `profit[i] + dp[i-1][c-weight[i]]`

Finally, our optimal solution will be maximum of the above two values:

```
dp[i][c] = max (dp[i-1][c], profit[i] + dp[i-1][c-weight[i]])
```

Let's draw this visually and start with our base case of zero capacity:



Code

Here is the code for our bottom-up dynamic programming approach:

Java | Python3 | C++ | JS

```
class Knapsack {

  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    // basic checks
    if (capacity <= 0 || profits.length == 0 || weights.length != profits.length)
      return 0;

    int n = profits.length;
    int[][] dp = new int[n][capacity + 1];

    // populate the capacity=0 columns, with '0' capacity we have '0' profit
    for(int i=0; i < n; i++)
      dp[i][0] = 0;
```

```
14
15      // if we have only one weight, we will take it if it is not more than the capacity
16      for(int c=0; c <= capacity; c++) {
17        if(weights[0] <= c)
18          dp[0][c] = profits[0];
19      }
20
21      // process all sub-arrays for all the capacities
22      for(int i=1; i < n; i++) {
23        for(int c=1; c <= capacity; c++) {
24          int profit1= 0, profit2 = 0;
25          // include the item, if it is not more than the capacity
26          if(weights[i] <= c)
27            profit1 = profits[i] + dp[i-1][c-weights[i]];
28          // exclude the item
29          profit2 = dp[i-1][c];
30          // take maximum
31          dp[i][c] = Math.max(profit1, profit2);
32        }
33      }
34
35      // maximum profit will be at the bottom-right corner.
36      return dp[n-1][capacity];
37    }
38
39    public static void main(String[] args) {
40      Knapsack ks = new Knapsack();
41      int[] profits = {1, 6, 10, 16};
42      int[] weights = {1, 2, 3, 5};
43      int maxProfit = ks.solveKnapsack(profits, weights, 7);
44      System.out.println("Total knapsack profit ---> " + maxProfit);
45      maxProfit = ks.solveKnapsack(profits, weights, 6);
46      System.out.println("Total knapsack profit ---> " + maxProfit);
47    }
48  }
```

Run Save Reset

Time and Space complexity

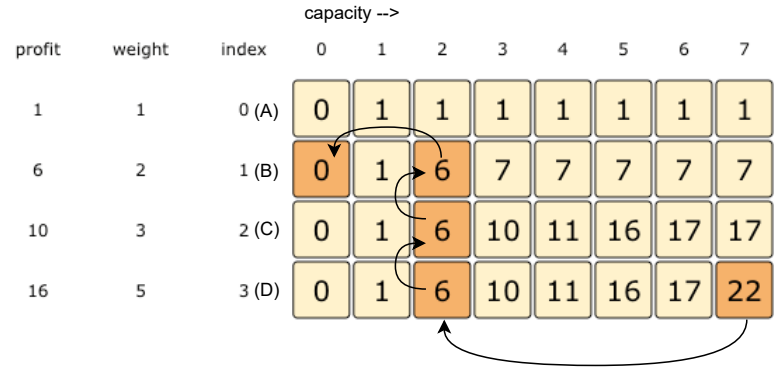
The above solution has the time and space complexity of $O(N * C)$, where 'N' represents total items, and 'C' is the maximum capacity.

How can we find the selected items?

As we know, the final profit is at the bottom-right corner. Therefore, we will start from there to find the items that will be going into the knapsack.

As you remember, at every step, we had two options: include an item or skip it. If we skip an item, we take the profit from the remaining items (i.e., from the cell right above it); if we include the item, then we jump to the remaining profit to find more items.

Let's understand this from the above example:



1. ‘22’ did not come from the top cell (which is 17); hence we must include the item at index ‘3’ (which is item ‘D’).
2. Subtract the profit of item ‘D’ from ‘22’ to get the remaining profit ‘6’. We then jump to profit ‘6’ on the same row.
3. ‘6’ came from the top cell, so we jump to row ‘2’.
4. Again, ‘6’ came from the top cell, so we jump to row ‘1’.
5. ‘6’ is different from the top cell, so we must include this item (which is item ‘B’).
6. Subtract the profit of ‘B’ from ‘6’ to get profit ‘0’. We then jump to profit ‘0’ on the same row. As soon as we hit zero remaining profit, we can finish our item search.
7. Thus, the items going into the knapsack are {B, D}.

Let’s write a function to print the set of items included in the knapsack.

Java | Python3 | C++ | JS

```java
import java.util.*;

class Knapsack {

  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    // base checks
    if (capacity <= 0 || profits.length == 0 || weights.length != profits.length)
      return 0;

    int n = profits.length;
    int[][] dp = new int[n][capacity + 1];

    // populate the capacity=0 columns, with '0' capacity we have '0' profit
    for(int i=0; i < n; i++)
      dp[i][0] = 0;

    // if we have only one weight, we will take it if it is not more than the capacity
    for(int c=0; c <= capacity; c++) {
      if(weights[0] <= c)
        dp[0][c] = profits[0];
    }

    // process all sub-arrays for all the capacities
    for(int i=1; i < n; i++) {
      for(int c=1; c <= capacity; c++) {
        int profit1= 0, profit2 = 0;
        // include the item, if it is not more than the capacity
        if(weights[i] <= c)
          profit1 = profits[i] + dp[i-1][c-weights[i]];
        // exclude the item
        profit2 = dp[i-1][c];
        // take maximum
        dp[i][c] = Math.max(profit1, profit2);
      }
    }
```

```
    }

    printSelectedElements(dp, weights, profits, capacity);
    // maximum profit will be at the bottom-right corner.
    return dp[n-1][capacity];
  }

 private void printSelectedElements(int dp[][], int[] weights, int[] profits, int capacity){
   System.out.print("Selected weights:");
   int totalProfit = dp[weights.length-1][capacity];
   for(int i=weights.length-1; i > 0; i--) {
     if(totalProfit != dp[i-1][capacity]) {
       System.out.print(" " + weights[i]);
       capacity -= weights[i];
       totalProfit -= profits[i];
     }
   }

   if(totalProfit != 0)
     System.out.print(" " + weights[0]);
   System.out.println("");
 }

  public static void main(String[] args) {
    Knapsack ks = new Knapsack();
    int[] profits = {1, 6, 10, 16};
    int[] weights = {1, 2, 3, 5};
    int maxProfit = ks.solveKnapsack(profits, weights, 7);
    System.out.println("Total knapsack profit ---> " + maxProfit);
    maxProfit = ks.solveKnapsack(profits, weights, 6);
    System.out.println("Total knapsack profit ---> " + maxProfit);
  }
}
```

Run Save Reset

Challenge

Can we improve our bottom-up DP solution even further? Can you find an algorithm that has $O(C)$ space complexity?

Show Hint

Java Python3 C++ JS

```
class Knapsack {

  static int solveKnapsack(int[] profits, int[] weights, int capacity) {
    //TODO: Write - Your - Code
    return -1;
  }
}
```

Test Show Solution Save Reset

The solution above is similar to the previous solution; the only difference is that we use `i%2` instead of `i` and `(i-1)%2` instead of `i-1`. This solution has a space complexity of $O(2 * C) = O(C)$, where 'C' is the knapsack's maximum capacity.

This space optimization solution can also be implemented using a single array. It is a bit tricky, but the intuition is to use the same array for the previous and the next iteration!

If you see closely, we need two values from the previous iteration: `dp[c]` and `dp[c-weight[i]]`

Since our inner loop is iterating over `c:0-->capacity`, let's see how this might affect our two required values:

1. When we access `dp[c],` it has not been overridden yet for the current iteration, so it should be fine.
2. `dp[c-weight[i]]` might be overridden if "weight[i] > 0". Therefore we can't use this value for the current iteration.

To solve the second case, we can change our inner loop to process in the reverse direction: `c:capacity-->0`. This will ensure that whenever we change a value in `dp[],` we will not need it again in the current iteration.

Can you try writing this algorithm?

Java | Python3 | C++ | JS

```
class Knapsack {

  static int solveKnapsack(int[] profits, int[] weights, int capacity) {
    //TODO: Write - Your - Code
    return -1;
  }
}
```

Equal Subset Sum Partition (medium)

We'll cover the following

- Problem Statement
- Try it yourself
- Basic Solution
  - Code
  - Time and Space complexity
- Top-down Dynamic Programming with Memoization
  - Code
  - Time and Space complexity
- Bottom-up Dynamic Programming
  - Code
  - Time and Space complexity

Problem Statement

Given a set of positive numbers, find if we can partition it into two subsets such that the sum of elements in both subsets is equal.

Example 1:

```
Input: {1, 2, 3, 4}
Output: True
Explanation: The given set can be partitioned into two subsets with equal sum: {1, 4} & {2, 3}
```

Example 2:

```
Input: {1, 1, 3, 4, 7}
Output: True
Explanation: The given set can be partitioned into two subsets with equal sum: {1, 3, 4} & {1, 7}
```

Example 3:

```
Input: {2, 3, 4, 6}
Output: False
Explanation: The given set cannot be partitioned into two subsets with equal sum.
```

Try it yourself

This problem looks similar to the 0/1 Knapsack problem. Try solving it before moving on to see the solution:

Java | Python3 | JS | C++

```
class PartitionSet {

  static boolean canPartition(int[] num) {
    //TODO: Write - Your - Code
    return false;
  }
}
```

Test | Save | Reset

Basic Solution

This problem follows the **0/1 Knapsack pattern**. A basic brute-force solution could be to try all combinations of partitioning the given numbers into two sets to see if any pair of sets has an equal sum.

Assume that `S` represents the total sum of all the given numbers. Then the two equal subsets must have a sum equal to `S/2`. This essentially transforms our problem to: "Find a subset of the given numbers that has a total sum of `S/2`".

So our brute-force algorithm will look like:

```
for each number 'i'
  create a new set which INCLUDES number 'i' if it does not exceed 'S/2', and recursively
      process the remaining numbers
  create a new set WITHOUT number 'i', and recursively process the remaining items
return true if any of the above sets has a sum equal to 'S/2', otherwise return false
```

Code

Here is the code for the brute-force solution:

Java | Python3 | C++ | JS

```
class PartitionSet {

  public boolean canPartition(int[] num) {
    int sum = 0;
    for (int i = 0; i < num.length; i++)
      sum += num[i];

    // if 'sum' is a an odd number, we can't have two subsets with equal sum
    if(sum % 2 != 0)
      return false;

    return this.canPartitionRecursive(num, sum/2, 0);
  }

  private boolean canPartitionRecursive(int[] num, int sum, int currentIndex) {
    // base check
    if (sum == 0)
      return true;

    if(num.length == 0 || currentIndex >= num.length)
      return false;

    // recursive call after choosing the number at the currentIndex
```

```
    // if the number at currentIndex exceeds the sum, we shouldn't process this
    if( num[currentIndex] <= sum ) {
      if(canPartitionRecursive(num, sum - num[currentIndex], currentIndex + 1))
        return true;
    }

    // recursive call after excluding the number at the currentIndex
    return canPartitionRecursive(num, sum, currentIndex + 1);
  }

  public static void main(String[] args) {
    PartitionSet ps = new PartitionSet();
    int[] num = {1, 2, 3, 4};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 1, 3, 4, 7};
    System.out.println(ps.canPartition(num));
    num = new int[]{2, 3, 4, 6};
    System.out.println(ps.canPartition(num));
  }
}
```

Run Save Reset

Time and Space complexity

The time complexity of the above algorithm is exponential $O(2^n)$, where 'n' represents the total number. The space complexity is $O(n)$, which will be used to store the recursion stack.

Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems. As stated in previous lessons, memoization is when we store the results of all the previously solved sub-problems so we can return the results from memory if we encounter a problem that has already been solved.

Since we need to store the results for every subset and for every possible sum, therefore we will be using a two-dimensional array to store the results of the solved sub-problems. The first dimension of the array will represent different subsets and the second dimension will represent different 'sums' that we can calculate from each subset. These two dimensions of the array can also be inferred from the two changing values (sum and currentIndex) in our recursive function `canPartitionRecursive()`.

Code

Here is the code for Top-down DP with memoization:

Java Python3 C++ JS

```
class PartitionSet {

  public boolean canPartition(int[] num) {
    int sum = 0;
    for (int i = 0; i < num.length; i++)
      sum += num[i];

    // if 'sum' is a an odd number, we can't have two subsets with equal sum
    if (sum % 2 != 0)
      return false;
```

```
    Boolean[][] dp = new Boolean[num.length][sum / 2 + 1];
    return this.canPartitionRecursive(dp, num, sum / 2, 0);
  }

  private boolean canPartitionRecursive(Boolean[][] dp, int[] num, int sum, int currentIndex) {
    // base check
    if (sum == 0)
      return true;

    if (num.length == 0 || currentIndex >= num.length)
      return false;

    // if we have not already processed a similar problem
    if (dp[currentIndex][sum] == null) {
      // recursive call after choosing the number at the currentIndex
      // if the number at currentIndex exceeds the sum, we shouldn't process this
      if (num[currentIndex] <= sum) {
        if (canPartitionRecursive(dp, num, sum - num[currentIndex], currentIndex + 1)) {
          dp[currentIndex][sum] = true;
          return true;
        }
      }

      // recursive call after excluding the number at the currentIndex
      dp[currentIndex][sum] = canPartitionRecursive(dp, num, sum, currentIndex + 1);
    }

    return dp[currentIndex][sum];
  }

  public static void main(String[] args) {
    PartitionSet ps = new PartitionSet();
    int[] num = { 1, 2, 3, 4 };
    System.out.println(ps.canPartition(num));
    num = new int[] { 1, 1, 3, 4, 7 };
    System.out.println(ps.canPartition(num));
    num = new int[] { 2, 3, 4, 6 };
    System.out.println(ps.canPartition(num));
  }
}
```

Run Save Reset

Time and Space complexity

The above algorithm has the time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the total sum of all the numbers.

Bottom-up Dynamic Programming

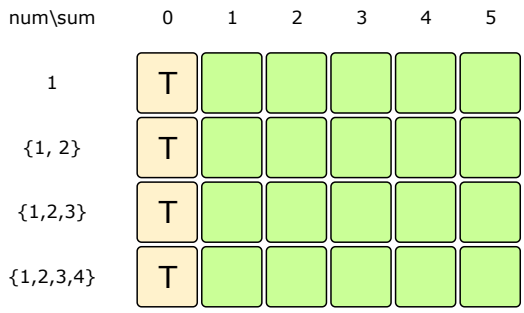
Let's try to populate our `dp[][]` array from the above solution by working in a bottom-up fashion. Essentially, we want to find if we can make all possible sums with every subset. **This means, `dp[i][s]` will be 'true' if we can make the sum 's' from the first 'i' numbers.**

So, for each number at index 'i' (0 <= i < num.length) and sum 's' (0 <= s <= S/2), we have two options:

1. Exclude the number. In this case, we will see if we can get 's' from the subset excluding this number: `dp[i-1][`
2. Include the number if its value is not more than 's'. In this case, we will see if we can find a subset to get the remaining sum: `dp[i-1][s-num[i]]`

If either of the two above scenarios is true, we can find a subset of numbers with a sum equal to ‘s’.

Let’s start with our base case of zero capacity:



'0' sum can always be found through an empty set

1 of 10

From the above visualization, we can clearly see that it is possible to partition the given set into two subsets with equal sums, as shown by bottom-right cell: `dp[3][5] => T`

Code

Here is the code for our bottom-up dynamic programming approach:

Java | Python3 | C++ | JS

```
class PartitionSet {

  public boolean canPartition(int[] num) {
    int n = num.length;
    // find the total sum
    int sum = 0;
    for (int i = 0; i < n; i++)
      sum += num[i];

    // if 'sum' is a an odd number, we can't have two subsets with same total
    if(sum % 2 != 0)
      return false;

    // we are trying to find a subset of given numbers that has a total sum of ‘sum/2’.
    sum /= 2;

    boolean[][] dp = new boolean[n][sum + 1];

    // populate the sum=0 columns, as we can always for '0' sum with an empty set
    for(int i=0; i < n; i++)
      dp[i][0] = true;

    // with only one number, we can form a subset only when the required sum is equal to its value
    for(int s=1; s <= sum ; s++) {
      dp[0][s] = (num[0] == s ? true : false);
    }

    // process all subsets for all sums
```

```
    for(int i=1; i < n; i++) {
      for(int s=1; s <= sum; s++) {
        // if we can get the sum 's' without the number at index 'i'
        if(dp[i-1][s]) {
          dp[i][s] = dp[i-1][s];
        } else if (s >= num[i]) { // else if we can find a subset to get the remaining sum
          dp[i][s] = dp[i-1][s-num[i]];
        }
      }
    }

    // the bottom-right corner will have our answer.
    return dp[n-1][sum];
  }

  public static void main(String[] args) {
    PartitionSet ps = new PartitionSet();
    int[] num = {1, 2, 3, 4};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 1, 3, 4, 7};
    System.out.println(ps.canPartition(num));
    num = new int[]{2, 3, 4, 6};
    System.out.println(ps.canPartition(num));
  }
}
```

Run Save Reset

Time and Space complexity

The above solution the has time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the total sum of all the numbers.

Back

0/1 Knapsack (medium)

Mark As Completed Next

Subset Sum (medium)

Subset Sum (medium)

We'll cover the following

- Problem Statement
  - Example 1:
  - Example 2:
  - Example 3:
- Try it yourself
- Basic Solution
- Bottom-up Dynamic Programming
  - Code
  - Time and Space complexity
- Challenge

Problem Statement

Given a set of positive numbers, determine if a subset exists whose sum is equal to a given number ‘S’.

Example 1:

```
Input: {1, 2, 3, 7}, S=6
Output: True
The given set has a subset whose sum is '6': {1, 2, 3}
```

Example 2:

```
Input: {1, 2, 7, 1, 5}, S=10
Output: True
The given set has a subset whose sum is '10': {1, 2, 7}
```

Example 3:

```
Input: {1, 3, 4, 8}, S=6
Output: False
The given set does not have any subset whose sum is equal to '6'.
```

Try it yourself

Try solving this question here:

Java | Python3 | C++ | JS

```
1  class SubsetSum {
```

```
2
3    public boolean canPartition(int[] num, int sum) {
4      // TODO: Write your code here
5      return false;
6    }
7
8    public static void main(String[] args) {
9      SubsetSum ss = new SubsetSum();
10     int[] num = { 1, 2, 3, 7 };
11     System.out.println(ss.canPartition(num, 6));
12     num = new int[] { 1, 2, 7, 1, 5 };
13     System.out.println(ss.canPartition(num, 10));
14     num = new int[] { 1, 3, 4, 8 };
15     System.out.println(ss.canPartition(num, 6));
16   }
17 }
```

Run Save Reset

Basic Solution

This problem follows the **0/1 Knapsack pattern** and is quite similar to Equal Subset Sum Partition. A basic brute-force solution could be to try all subsets of the given numbers to see if any set has a sum equal to 'S'.

So our brute-force algorithm will look like:

```
for each number 'i'
  create a new set which INCLUDES number 'i' if it does not exceed 'S', and recursively
     process the remaining numbers
  create a new set WITHOUT number 'i', and recursively process the remaining numbers
return true if any of the above two sets has a sum equal to 'S', otherwise return false
```

Since this problem is quite similar to Equal Subset Sum Partition, let's jump directly to the bottom-up dynamic programming solution.

Bottom-up Dynamic Programming

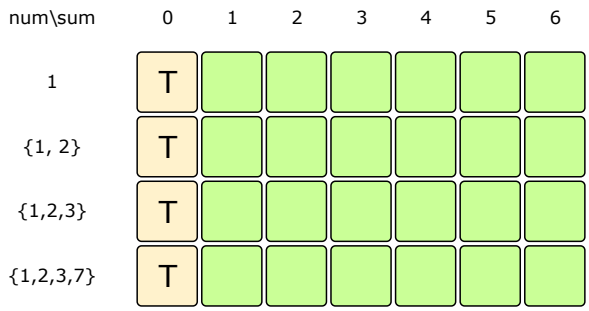
We'll try to find if we can make all possible sums with every subset to populate the array `dp[TotalNumbers][S+1]`.

For every possible sum 's' (where 0 <= s <= S), we have two options:

1. Exclude the number. In this case, we will see if we can get the sum 's' from the subset excluding this number => `dp[index-1][s]`
2. Include the number if its value is not more than 's'. In this case, we will see if we can find a subset to get the remaining sum => `dp[index-1][s-num[index]]`

If either of the above two scenarios returns true, we can find a subset with a sum equal to 's'.

Let's draw this visually, with the example input {1, 2, 3, 7}, and start with our base case of size zero:



'0' sum can always be found through an empty set

1 of 10

Code

Here is the code for our bottom-up dynamic programming approach:

Java | Python3 | C++ | JS

```
class SubsetSum {

  public boolean canPartition(int[] num, int sum) {
    int n = num.length;
    boolean[][] dp = new boolean[n][sum + 1];

    // populate the sum=0 columns, as we can always form '0' sum with an empty set
    for (int i = 0; i < n; i++)
      dp[i][0] = true;

    // with only one number, we can form a subset only when the required sum is
    // equal to its value
    for (int s = 1; s <= sum; s++) {
      dp[0][s] = (num[0] == s ? true : false);
    }

    // process all subsets for all sums
    for (int i = 1; i < num.length; i++) {
      for (int s = 1; s <= sum; s++) {
        // if we can get the sum 's' without the number at index 'i'
        if (dp[i - 1][s]) {
          dp[i][s] = dp[i - 1][s];
        } else if (s >= num[i]) {
          // else include the number and see if we can find a subset to get the remaining
          // sum
          dp[i][s] = dp[i - 1][s - num[i]];
        }
      }
    }

    // the bottom-right corner will have our answer.
    return dp[num.length - 1][sum];
  }

  public static void main(String[] args) {
    SubsetSum ss = new SubsetSum();
    int[] num = { 1, 2, 3, 7 };
```

```
38      System.out.println(ss.canPartition(num, 6));
39      num = new int[] { 1, 2, 7, 1, 5 };
40      System.out.println(ss.canPartition(num, 10));
41      num = new int[] { 1, 3, 4, 8 };
42      System.out.println(ss.canPartition(num, 6));
43    }
44  }
```

Run Save Reset

Time and Space complexity

The above solution has the time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the required sum.

Challenge

Can we improve our bottom-up DP solution even further? Can you find an algorithm that has $O(S)$ space complexity?

Show Hint

Java Python3 C++ JS

```
1  class SubsetSum {
2
3    static boolean canPartition(int[] num, int sum) {
4      //TODO: Write - Your - Code
5      return false;
6    }
7  }
```

Test Show Solution Save Reset

Back

Equal Subset Sum Partition (medium)

Mark As Completed Next

Minimum Subset Sum Difference (hard)

Minimum Subset Sum Difference (hard)

We'll cover the following

- Problem Statement
  - Example 1:
  - Example 2:
  - Example 3:
- Try it yourself
- Basic Solution
  - Code
  - Time and Space complexity
- Top-down Dynamic Programming with Memoization
  - Code
  - Bottom-up Dynamic Programming
    - Code
  - Time and Space complexity

Problem Statement

Given a set of positive numbers, partition the set into two subsets with minimum difference between their subset sums.

Example 1:

```
Input: {1, 2, 3, 9}
Output: 3
Explanation: We can partition the given set into two subsets where minimum absolute difference
between the sum of numbers is '3'. Following are the two subsets: {1, 2, 3} & {9}.
```

Example 2:

```
Input: {1, 2, 7, 1, 5}
Output: 0
Explanation: We can partition the given set into two subsets where minimum absolute difference
between the sum of number is '0'. Following are the two subsets: {1, 2, 5} & {7, 1}.
```

Example 3:

```
Input: {1, 3, 100, 4}
Output: 92
```

```
Explanation: We can partition the given set into two subsets where minimum absolute difference
between the sum of numbers is '92'. Here are the two subsets: {1, 3, 4} & {100}.
```

Try it yourself

Try solving this question here:

Java | Python3 | C++ | JS

```
class PartitionSet {

  public int canPartition(int[] num) {
    // TODO: Write your code here
    return -1;
  }

  public static void main(String[] args) {
    PartitionSet ps = new PartitionSet();
    int[] num = {1, 2, 3, 9};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 2, 7, 1, 5};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 3, 100, 4};
    System.out.println(ps.canPartition(num));
  }
}
```

Run | Save | Reset

Basic Solution

This problem follows the **0/1 Knapsack pattern** and can be converted into a Subset Sum problem.

Let's assume S1 and S2 are the two desired subsets. A basic brute-force solution could be to try adding each element either in S1 or S2 in order to find the combination that gives the minimum sum difference between the two sets.

So our brute-force algorithm will look like:

```
for each number 'i'
  add number 'i' to S1 and recursively process the remaining numbers
  add number 'i' to S2 and recursively process the remaining numbers
return the minimum absolute difference of the above two sets
```

Code

Here is the code for the brute-force solution:

Java | Python3 | C++ | JS

```
class PartitionSet {

  public int canPartition(int[] num) {
    return this.canPartitionRecursive(num, 0, 0, 0);
  }

  private int canPartitionRecursive(int[] num, int currentIndex, int sum1, int sum2) {
    // base check
```

```
    // base check
    if (currentIndex == num.length)
      return Math.abs(sum1 - sum2);

    // recursive call after including the number at the currentIndex in the first set
    int diff1 = canPartitionRecursive(num, currentIndex+1, sum1+num[currentIndex], sum2);

    // recursive call after including the number at the currentIndex in the second set
    int diff2 = canPartitionRecursive(num, currentIndex+1, sum1, sum2+num[currentIndex]);

    return Math.min(diff1, diff2);
  }

  public static void main(String[] args) {
    PartitionSet ps = new PartitionSet();
    int[] num = {1, 2, 3, 9};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 2, 7, 1, 5};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 3, 100, 4};
    System.out.println(ps.canPartition(num));
  }
}
```

Run Save Reset

Time and Space complexity

Because of the two recursive calls, the time complexity of the above algorithm is exponential $O(2^n)$, where 'n' represents the total number. The space complexity is $O(n)$ which is used to store the recursion stack.

Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems.

We will be using a two-dimensional array to store the results of the solved sub-problems. We can uniquely identify a sub-problem from 'currentIndex' and 'Sum1' as 'Sum2' will always be the sum of the remaining numbers.

Code

Here is the code:

Java Python3 C++ JS

```
class PartitionSet {

  public int canPartition(int[] num) {
    int sum = 0;
    for (int i = 0; i < num.length; i++)
      sum += num[i];

    Integer[][] dp = new Integer[num.length][sum + 1];
    return this.canPartitionRecursive(dp, num, 0, 0, 0);
  }

  private int canPartitionRecursive(Integer[][] dp, int[] num, int currentIndex, int sum1, int sum2) {
    // base check
    if(currentIndex == num.length)
      return Math.abs(sum1 - sum2);

    // check if we have not already processed similar problem
```

```
    if(dp[currentIndex][sum1] == null) {
      // recursive call after including the number at the currentIndex in the first set
      int diff1 = canPartitionRecursive(dp, num, currentIndex + 1, sum1 + num[currentIndex], sum2);

      // recursive call after including the number at the currentIndex in the second set
      int diff2 = canPartitionRecursive(dp, num, currentIndex + 1, sum1, sum2 + num[currentIndex]);

      dp[currentIndex][sum1] = Math.min(diff1, diff2);
    }

    return dp[currentIndex][sum1];
  }

  public static void main(String[] args) {
    PartitionSet ps = new PartitionSet();
    int[] num = {1, 2, 3, 9};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 2, 7, 1, 5};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 3, 100, 4};
    System.out.println(ps.canPartition(num));
  }
}
```

Run Save Reset

Bottom-up Dynamic Programming

Let's assume 'S' represents the total sum of all the numbers. So, in this problem, we are trying to find a subset whose sum is as close to 'S/2' as possible, because if we can partition the given set into two subsets of an equal sum, we get the minimum difference, i.e. zero. This transforms our problem to Subset Sum, where we try to find a subset whose sum is equal to a given number-- 'S/2' in our case. If we can't find such a subset, then we will take the subset which has the sum closest to 'S/2'. This is easily possible, as we will be calculating all possible sums with every subset.

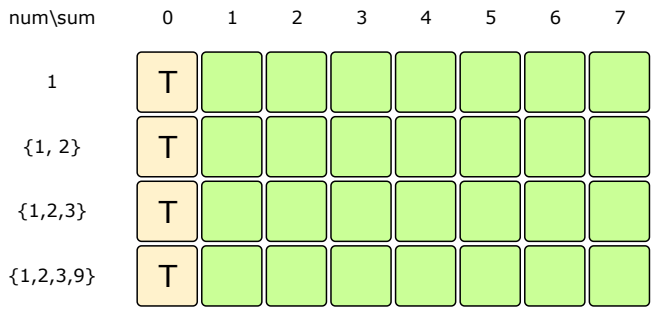
Essentially, we need to calculate all the possible sums up to 'S/2' for all numbers. So how can we populate the array `db[TotalNumbers][S/2+1]` in the bottom-up fashion?

For every possible sum 's' (where 0 <= s <= S/2), we have two options:

1. Exclude the number. In this case, we will see if we can get the sum 's' from the subset excluding this number => `dp[index-1][s]`
2. Include the number if its value is not more than 's'. In this case, we will see if we can find a subset to get the remaining sum => `dp[index-1][s-num[index]]`

If either of the two above scenarios is true, we can find a subset with a sum equal to 's'. We should dig into this before we can learn how to find the closest subset.

Let's draw this visually, with the example input {1, 2, 3, 9}. Since the total sum is '15', we will try to find a subset whose sum is equal to the half of it, i.e. '7'.



'0' sum can always be found through an empty set

1 of 11

The above visualization tells us that it is not possible to find a subset whose sum is equal to ‘7’. So what is the closest subset we can find? We can find the subset if we start moving backwards in the last row from the bottom right corner to find the first ‘T’. The first “T” in the diagram above is the sum ‘6’, which means that we can find a subset whose sum is equal to ‘6’. This means the other set will have a sum of ‘9’ and the minimum difference will be ‘3’.

Code

Here is the code for our bottom-up dynamic programming approach:

Java | Python3 | C++ | JS

```
class PartitionSet {

  public int canPartition(int[] num) {
    int sum = 0;
    for (int i = 0; i < num.length; i++)
      sum += num[i];

    int n = num.length;
    boolean[][] dp = new boolean[n][sum/2 + 1];

    // populate the sum=0 columns, as we can always form '0' sum with an empty set
    for(int i=0; i < n; i++)
      dp[i][0] = true;

    // with only one number, we can form a subset only when the required sum is equal to that number
    for(int s=1; s <= sum/2 ; s++) {
      dp[0][s] = (num[0] == s ? true : false);
    }

    // process all subsets for all sums
    for(int i=1; i < num.length; i++) {
      for(int s=1; s <= sum/2; s++) {
        // if we can get the sum 's' without the number at index 'i'
        if(dp[i-1][s]) {
          dp[i][s] = dp[i-1][s];
        } else if (s >= num[i]) {
          // else include the number and see if we can find a subset to get the remaining sum
          dp[i][s] = dp[i-1][s-num[i]];
        }
      }
```

```
    }

    int sum1 = 0;
    // Find the largest index in the last row which is true
    for (int i = sum / 2; i >= 0; i--) {
      if (dp[n-1][i] == true) {
          sum1 = i;
          break;
      }
    }

    int sum2 = sum - sum1;
    return Math.abs(sum2-sum1);
  }

  public static void main(String[] args) {
    PartitionSet ps = new PartitionSet();
    int[] num = {1, 2, 3, 9};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 2, 7, 1, 5};
    System.out.println(ps.canPartition(num));
    num = new int[]{1, 3, 100, 4};
    System.out.println(ps.canPartition(num));
  }
}
```

Run Save Reset

Time and Space complexity

The above solution has the time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the total sum of all the numbers.

Problem Challenge 1

We'll cover the following

- Count of Subset Sum (hard)
  - Example 1:
  - Example 2:
- Try it yourself

Count of Subset Sum (hard)

Given a set of positive numbers, find the total number of subsets whose sum is equal to a given number ‘S’.

Example 1:

```
Input: {1, 1, 2, 3}, S=4
Output: 3
The given set has '3' subsets whose sum is '4': {1, 1, 2}, {1, 3}, {1, 3}
Note that we have two similar sets {1, 3}, because we have two '1' in our input.
```

Example 2:

```
Input: {1, 2, 7, 1, 5}, S=9
Output: 3
The given set has '3' subsets whose sum is '9': {2, 7}, {1, 7, 1}, {1, 2, 1, 5}
```

Try it yourself

Try solving this question here:

Java | Python3 | C++ | JS

```java
class SubsetSum {
  static int countSubsets(int[] num, int sum) {
    //TODO: Write - Your - Code
    return -1;
  }

  public static void main(String[] args) {
    SubsetSum ss = new SubsetSum();
    int[] num = { 1, 1, 2, 3 };
    System.out.println(ss.countSubsets(num, 4));
    num = new int[] { 1, 2, 7, 1, 5 };
    System.out.println(ss.countSubsets(num, 9));
  }
}
```

Run | Show Solution | Save | Reset

Back | Mark As Completed | Next

Minimum Subset Sum Difference (hard)

Solution Review: Problem Challenge 1

educative

Problem Challenge 2

We'll cover the following

- Target Sum (hard)
  - Example 1:
  - Example 2:
- Try it yourself

Target Sum (hard)

You are given a set of positive numbers and a target sum ‘S’. Each number should be assigned either a ‘+’ or ‘-’ sign. We need to find the total ways to assign symbols to make the sum of the numbers equal to the target ‘S’.

Example 1:

```
Input: {1, 1, 2, 3}, S=1
Output: 3
Explanation: The given set has '3' ways to make a sum of '1': {+1-1-2+3} & {-1+1-2
+3} & {+1+1+2-3}
```

Example 2:

```
Input: {1, 2, 7, 1}, S=9
Output: 2
Explanation: The given set has '2' ways to make a sum of '9': {+1+2+7-1} & {-1+2+7
+1}
```

Try it yourself

Try solving this question here:

Java | Python3 | C++ | JS

```java
class TargetSum {

  public int findTargetSubsets(int[] num, int s) {
    // TODO: Write your code here
    return -1;
  }

  public static void main(String[] args) {
    TargetSum ts = new TargetSum();
    int[] num = {1, 1, 2, 3};
    System.out.println(ts.findTargetSubsets(num, 1));
    num = new int[]{1, 2, 7, 1};
    System.out.println(ts.findTargetSubsets(num, 9));
  }
}

```

Run Save Reset

Back Mark As Completed Next

Solution Review: Problem Challenge 1 Solution Review: Problem Challenge 2

educative